AI 투자자 기반 가상 주식시장 모의투자 대회 플랫폼

1차 설계제안서

제출일	2026.10.12	**소속학부**	소프트웨어학부
팀명	hamburger		
팀원	김영재, 김경주, 이경하, 이윤성, 박재호		

1. 프로젝트 소개

본 프로젝트는 실제 시세를 가져오지 않고, 가상 기업으로 이루어진 자체 주식시장을 직접 설계·운영하는 모의투자 대회 플랫폼이다. 기존 모의투자 서비스는 실제 시세를 그대로 따라가기 때문에 사용자의 주문이 가격에 영향을 주지 못한다[1]. 본 플랫폼에서는 사용자의 주문이 자체 체결 엔진에서 실제로 체결되고, 개미·기관·외국인 등 서로 다른 성향의 AI 투자자(봇)가 함께 거래하면서 수요와 공급에 따라 가격이 형성된다.

시장에는 금리, 실적 발표, 자연재해, 정부 정책, 전쟁과 같은 사건이 발생하며, 봇들은 이를 각자의 성향에 따라 다르게 해석하여 매매한다. 사용자는 하나의 가상 주식장에서 투자 활동을 하고, 1개월·3개월·6개월·1년 단위의 기간별 대회에서 해당 기간의 투자 성과로 순위를 겨룬다. 가상 기업·가상 화폐·가상 사건만 사용하며, 실제 금융 상품의 거래나 투자 권유와는 무관하다.

※ 본문의 [숫자]는 7장 참고문헌의 번호이다.

2. 기존 시스템 분석

2-1. 참고 대상 분석

- 증권사 모의투자 서비스[1]
  - ○ 실제 시세에 맞춰 가상 자산을 사고판다. 사용자 주문이 가격에 영향을 주지 않는다.
  - ○ 본 프로젝트는 플랫폼 내부 체결로 가격이 형성된다는 점에서 다르다.
- 한국거래소 투자자별 거래실적[3]
  - ○ 개인·기관·외국인 등 투자자 유형별 거래 통계를 공개한다.
  - ○ AI 투자자 봇의 페르소나 구분과 행동 파라미터 설계의 참고 자료로 활용한다.
- ABIDES (에이전트 기반 시장 시뮬레이터)[4]
  - ○ 여러 유형의 거래 에이전트가 거래소 에이전트를 통해 거래하는 이산 사건 기반 시장 시뮬레이터이다.
  - ○ 투자자 유형별 행동 설계와 시장 사실성 검증 방법을 참고한다.
- 한국거래소 매매 방식[2]
  - ○ 정규장은 연속매매이며 가격 우선·시간 우선 원칙으로 체결하고, 지정가·시장가 등 주문 유형이 있다.
  - ○ C 체결 엔진의 매칭 규칙으로 구현한다.
- 지정가 주문 장부(Limit Order Book) 연구[5]
  - ○ 주문 장부의 구조와 통계적 특성, 이를 설명하는 모형을 정리한 연구이다.
  - ○ 호가창 자료구조와 체결 엔진 설계, 시장 검증 지표의 근거로 활용한다.

2-2. 기존 서비스의 한계점 및 보완 방안

- 가격 형성 과정 체험 불가
  - 기존 모의투자는 사용자 주문이 가격에 영향을 주지 않는다. 자체 체결 엔진에서 사용자와 봇의 주문이 함께 체결되어 가격이 형성되도록 한다.
- 시장 상황 연출의 한계
  - 실제 시세에 종속되어 호황·불황·급변 같은 상황을 임의로 구성하기 어렵다. 가상 기업과 가상 사건으로 다양한 시장 상황을 구성한다.
- 뉴스·경제 상황의 가격 반영 연습 어려움
  - 사건이 봇의 매매 파라미터를 바꾸고, 같은 사건에도 투자자 유형별 반응이 달라지도록 하여 가격 반영 과정을 연습할 수 있게 한다.
- 참여 시점 및 일정 제한
  - 대회 개최 일정에 따라 참여가 제한된다. 하나의 시장에서 기간별 논리적 대회에 자동 편입되어 언제든 참여할 수 있도록 한다.

3. 특징 및 구상도

3-1. 특징

- 자체 가상 주식장
  - 외부 API 없이 종목, 장 운영 시간, 가격 규칙을 우리의 기준으로 설계한다.
- C 체결 엔진
  - 종목별 호가창을 관리하며 가격 우선·시간 우선으로 지정가·시장가 주문을 체결한다.
- AI 투자자 봇
  - 서로 다른 페르소나를 가진 봇이 백그라운드 스레드로 동작하며 시장 상태를 입력받아 주문을 낸다.
- LLM 기반 24시간 갱신
  - 24시간마다 외부 LLM API를 호출해 봇별 신규 파라미터와 사건 작용값을 받아오고, 검증 후 같은 주기로 시장에 반영한다.
- 사건 체계
  - 주가에 종속되는 사건과 종속되지 않는 사건을 구분하여 외부 충격과 내부 반응을 함께 만든다.
- 논리적 기간 대회
  - 별도 대회를 개설하지 않고, 기간별 투자 성과를 순위로 집계하는 논리적 단위로 운영한다.
- React 웹 클라이언트
  - 실시간 시세·호가·체결, 계좌, 기간별 랭킹을 화면으로 제공한다.

3-2. 정보구조도

3-2-1. 기능 단위

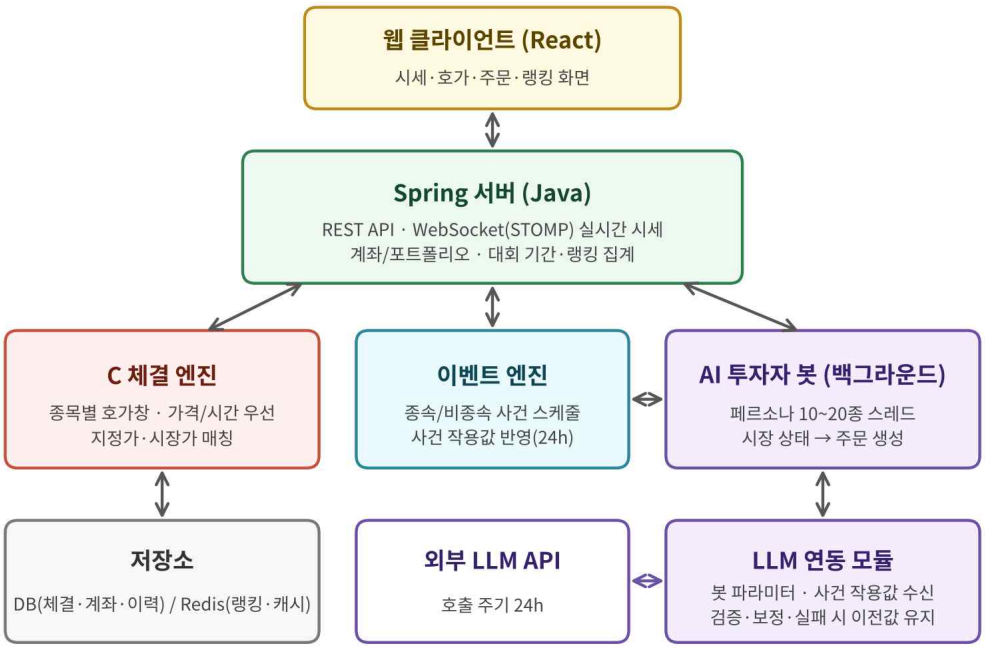


3-2-2. 페이지 단위

메인 페이지	종목 페이지	내 자산 페이지	대회 페이지	사건/뉴스 페이지	회원 페이지
시장 지수	실시간 차트	보유 종목	기간별 랭킹	사건 타임라인	회원가입/로그인
섹터 현황	호가창	평가손익	내 순위·수익률	사건 상세	회원정보 수정
종목 목록	주문 입력 (지정가/시장가)	주문 내역	지난 구간 결과	섹터별 영향	회원탈퇴/로그아웃

3-3. 기능 및 구상도

3-3-1. 시스템 구성도



구성 요소	기술	역할
웹 클라이언트	React[12]	시세·호가·주문·계좌·랭킹 화면, WebSocket 구독으로 실시간 갱신
체결 엔진	C	종목별 호가창 관리, 가격/시간 우선 매칭, 체결 결과 반환
메인 서버	Java, Spring[11]	REST API, WebSocket(STOMP) 실시간 시세, 계좌·포트폴리오, 대회 기간·랭킹 집계
AI 투자자 봇	Java 백그라운드 스레드	페르소나별 매매 판단 및 주문 생성, 파라미터 보관
이벤트 엔진	Java	사건 일정 관리, 종속/비종속 사건 발생, 24시간 주기 사건 작용값 반영
LLM 연동 모듈	외부 LLM API	프롬프트 구성, 응답 검증·보정, 실패 시 이전값 유지
저장소	관계형 DB, Redis[10]	체결·계좌·이력 영속 저장, 랭킹·시세 캐시

3-3-2. 가상 주식장 설계

- 종목: 반도체, IT, 금융, 통신 등 섹터별 가상 기업을 상장한다. 시장 단위로 엔진과 봇을 분리하여 이후 여러 시장으로 확장할 수 있도록 한다.
- 장 운영: 한국거래소 정규장의 매매 시간(09:00~15:30)을 기준으로 개장·폐장한다[2]. 시가·종가 단일가 매매는 확장 과제로 둔다.
- 가격제한폭: 한국거래소의 일일 가격제한폭(기준가격 대비 ±30%)[2]을 참고하여 적용하고, 봇의 과잉 반응으로 가격이 폭주하거나 붕괴하는 상황을 방지한다.
- 계좌: 가입 시 동일한 가상 시드머니를 지급하며, 하나의 계좌로 모든 기간 대회에 참여한다.

3-3-3. 체결 엔진 (C)

체결 엔진은 시장의 핵심이다. 성능과 메모리를 직접 제어하기 위해 C 언어로 구현한다.

- 주문 유형: 지정가, 시장가 / 주문 동작: 신규, 취소, 정정[2]
- 매칭 규칙: 가격 우선 → 시간 우선. 같은 가격에서는 먼저 들어온 주문이 먼저 체결된다[2][5].
- 자료구조: 종목별 매수·매도 호가창을 가격 레벨 단위로 관리하고, 레벨 내부는 FIFO 큐로 구성한다. 주문 ID로 즉시 취소할 수 있도록 해시 테이블을 함께 둔다.
- 동시성: 종목 단위로 호가창을 분리하여 종목별로 하나의 처리 흐름에서 순차 처리한다. 잠금 경합을 줄이고 결과를 재현 가능하게 만든다.
- 복구: 모든 주문·체결을 순서대로 로그로 남겨, 장애 후 로그를 재생하여 호가창을 복원한다.
- 서버 연동: Spring 서버와의 연동 방식(JNI[9], Java FFM API[8], 별도 프로세스 + 소켓/gRPC)은 성능과 개발 난이도를 비교한 뒤 개발 초기에 확정한다.

3-3-4. 논리적 기간 대회

대회는 미리 개설해 두는 이벤트가 아니라, 하나의 계좌와 하나의 시장 위에서 특정 기간만 떼어 투자 성과를 계산해 순위를 매기는 논리적 단위이다. 사용자가 시장에 참가하면 해당 시점의 기간 대회에 자동으로 편입된다.

대회 종류	산정 구간 (2027년 1월 참가 예시)	순위 기준
1개월 대회	2027.01.01 ~ 2027.01.31 (해당 연도 첫 번째 1개월)	구간 내 수익률
3개월 대회	2027.01.01 ~ 2027.03.31 (해당 연도 첫 번째 3개월)	구간 내 수익률
6개월 대회	2027.01.01 ~ 2027.06.30 (해당 연도 첫 번째 6개월)	구간 내 수익률
1년 대회	2027.01.01 ~ 2027.12.31 (해당 연도 첫 번째 1년)	구간 내 수익률

- 기간 수익률 = (구간 종료 시점 평가자산 − 구간 시작 시점 평가자산) ÷ 구간 시작 시점 평가자산. 시드머니 외 추가 입금은 없는 것을 기본으로 한다.
- 집계: 매 장 마감 시 계정별 평가자산을 스냅샷으로 저장하고, 진행 중인 구간의 순위는 Redis Sorted Set으로 갱신하여 빠르게 조회한다. Sorted Set은 점수 순 정렬을 유지하며 순위 조회와 갱신이 O(log N)이고, 동점은 멤버 사전순으로 구분된다[10].
- 구간 도중 참가: 참가 시점의 평가자산을 해당 구간의 기준 자산으로 삼아 진행 중인 구간에 바로 편입하고, 구간 종료 시점까지의 수익률로 순위를 매긴다.
- 구간이 끝나면 순위를 확정·보관하고, 다음 구간은 자동으로 시작된다.

3-3-5. AI 투자자 봇

봇은 백그라운드 스레드로 동작하는 자동 투자자이다. 약 10~20종의 서로 다른 페르소나를 두며, 각 봇은 현재 장 상황을 입력받아 자신의 파라미터에 따라 주문을 생성하고 체결 엔진에 전달한다. 사용자가 적은 초기에도 시장에 충분한 거래량과 호가가 유지되도록 하는 역할도 맡는다. 페르소나는 한국거래소의 투자자 유형 구분(개인·기관·외국인)[3]과 에이전트 기반 시뮬레이션의 에이전트 유형 설계[4]를 참고하여 구성한다.

페르소나 구성 예시 (초기 12종, 확장 가능)

구분	페르소나	성향
개인	단기 투자형	높은 거래 빈도, 짧은 보유, 뉴스·모멘텀에 민감
개인	장기 투자형	낮은 거래 빈도, 저평가 종목 분할 매수
개인	추세 추종형	상승 추세 종목을 추격 매수, 하락 시 빠르게 정리
개인	공포·군중 심리형	급락 시 투매, 급등 시 추격 매수
기관	연기금형 (국민연금 등)	대규모 자금을 장기·분산·분할 주문으로 운용, 낮은 회전율
기관	액티브 펀드형	섹터 로테이션, 실적·금리 변화에 민감
기관	배당·가치형	안정적인 종목 선호, 변동성 확대 시 비중 축소
기관	역추세(평균회귀)형	과열·과매도 구간에서 반대 방향으로 진입
외국인	패시브(지수 추종)형	시장 전체 비중을 따라 균형 매매
외국인	액티브형	환율·금리·거시 상황에 민감하게 반응
시장 조성	마켓메이커형	양방향 호가를 지속 공급하여 유동성 확보
시장 조성	노이즈 트레이더	소량의 무작위 주문으로 시장 활기 유지

- 투자 성향 파라미터: 위험 선호도, 목표 현금 비중, 종목·섹터별 최대 비중, 선호 섹터
- 매매 행태 파라미터: 거래 빈도, 평균 보유 기간, 주문당 규모, 지정가/시장가 비율, 지정가 호가 간격, 손절·익절 기준
- 신호 가중치: 추세, 평균회귀, 가치(저평가), 뉴스 반응의 가중치
- 사건 반응 파라미터: 종속 작용 민감도, 비종속 작용 민감도, 반응 지연 시간
- 입력: 종목별 현재가·호가·거래량·변동성, 섹터 수익률, 최근 사건, 자신의 보유 현황
- 출력: 주문(종목, 매수/매도, 수량, 지정가/시장가, 가격)을 체결 엔진에 전달
- 실행: 봇은 스레드 풀 위에서 각자의 주기로 동작하며, 주문 속도 제한을 두어 엔진에 과부하가 걸리지 않도록 한다.

3-3-6. LLM 연동과 24시간 갱신

LLM은 실시간 매매 판단에는 사용하지 않는다. 매매 판단은 서버 내부 로직이 담당하고, LLM은 하루 단위로 시장 전체의 흐름을 해석하여 봇의 성향 파라미터와 사건 작용값을 조정하는 역할만 맡는다. 봇 파라미터와 사건 작용값은 모두 24시간 주기의 한 번의 호출로 받아, 같은 주기로 한꺼번에 반영한다. 이렇게 하면 호출 비용과

응답 지연이 시장 처리 속도에 영향을 주지 않는다.

- 24시간 주기 호출: 프롬프트는 우리 프로젝트 환경(가상 시장 구조, 봇 페르소나와 파라미터 의미, 사건 체계, 출력 형식)에 대한 고정 설명 + 최근 24시간의 장 상황 요약(지수, 섹터 수익률, 거래량, 변동성, 최근 사건, 주가 히스토리) + 현재 봇 파라미터로 구성한다.
- 응답: 봇별 신규 파라미터 값과, 종속·비종속 사건에 대한 봇의 반응 파라미터를 정해진 JSON 형식으로 받는다.
- 검증과 보정: 형식 검사, 값 범위 보정, 1회 변경 폭 제한을 거친 값만 반영한다. 응답 실패나 형식 오류 시 재시도 후 이전 파라미터를 유지하고, 모든 호출의 프롬프트·응답·적용 여부를 저장하여 시장 변화를 추적하고 재현할 수 있게 한다.
- 신규 반영 비율: 신규 파라미터를 모든 봇에 한 번에 적용하면 시장이 급변할 수 있다. 각 페르소나를 여러 개의 봇 인스턴스로 구성하고, 전체 봇 중 신규 파라미터를 적용할 비율을 내부 설정값으로 두어 시장 성향이 점진적으로 이동하도록 한다. 이 비율은 시장 사실성 검증 결과[6][7]를 보며 조정한다.
- 24시간 주기 반영: 검증을 통과한 봇 파라미터와 사건 작용값을 신규 반영 비율에 따라 24시간마다 봇에 한꺼번에 반영한다. 비종속 사건의 일정은 연 1회 생성하고, 해당 사건에 대한 봇의 반응값은 매일의 24시간 호출에서 갱신된다.



3-3-7. 사건(이벤트) 체계

주식은 차트만으로 움직이지 않으므로, 각종 사건이 시장에 영향을 준다. 사건은 주가에 종속되는 작용과 종속되지 않는 작용으로 나눈다.

구분	예시	결정 방식
종속 작용	금리, 실적 발표 등	주가 히스토리를 바탕으로 LLM이 판단하여 생성
비종속 작용	자연재해, 정부 정책, 전쟁 등	연 1회 LLM이 1년치 일정(발생 월일, 유형, 영향 섹터, 강도, 지속 기간)을 미리 생성

- 실제 시장은 종속 작용이 많지만, 사건 설계를 단순하게 유지하기 위해 전체 사건의 절반 이상을 비종속 작용으로 구성한다.
- 봇 쪽에 종속 작용과 비종속 작용에 대한 반응 파라미터를 열어 두고, LLM이 24시간 호출에서 이 값을 함께 응답한다.
- 이 값은 봇 파라미터와 함께 24시간 주기로 한꺼번에 반영한다. 같은 사건도 페르소나별 민감도와 지연이 달라 서로 다른 매매와 가격 흐름이 만들어진다.
- 사건은 사용자에게 뉴스 형태로 노출한다. 루머·보도·공시처럼 단계적으로 공개하는 방식은 확장 과제로 둔다.

3-3-8. 주요 데이터

데이터	내용	저장 방식
종목·시장	종목 정보, 섹터, 상장 정보, 시장 설정	관계형 DB
주문·체결	주문 내역, 체결 내역(대용량, 지속 증가)	엔진 로그 + DB 배치 저장
시세	현재가, 호가 스냅샷, 분봉·일봉 집계	Redis(실시간) + DB(이력)
계좌·포지션	예수금, 보유 종목, 일별 평가자산 스냅샷	관계형 DB
대회·랭킹	기간 구간 정보, 구간별 수익률·순위	관계형 DB + Redis Sorted Set
봇 파라미터	페르소나별 파라미터 및 변경 이력	관계형 DB
사건	사건 일정, 유형, 영향 파라미터	관계형 DB
LLM 호출 기록	프롬프트, 응답, 검증 결과, 적용 여부	관계형 DB

3-3-9. 프론트엔드 (React)

웹 클라이언트는 React[12]로 구성한다. 계좌·랭킹·사건 조회는 REST API로 처리하고, 시세·호가·체결 알림은 WebSocket(STOMP)을 구독하여 화면을 갱신한다. WebSocket은 저지연·고빈도·대용량 메시지가 필요한 금융 서비스처럼 실시간성이 높은 화면에 적합하다[11].

- 화면 단위 컴포넌트: 시세 차트, 호가창, 주문 입력, 보유 종목 표, 기간별 랭킹 표, 사건 타임라인
- 실시간 데이터: 종목 상세 화면에 진입할 때 해당 종목의 시세·호가를 구독하고, 화면을 벗어나면 구독을 해제한다.

4. 간략수행계획서

아래와 같은 기능들을 체결 엔진, 서버·봇, LLM·이벤트, 프론트엔드·인프라로 나누어 구현한다.

- C 체결 엔진 구현
  - 종목별 호가창, 가격/시간 우선 매칭, 주문·체결 로그를 구현하고, Spring 서버와의 연동 방식(JNI[9], FFM API[8], 소켓/gRPC)을 확정한다.

- Spring 서버와 실시간 시세
  - 계좌·주문·시세 API와 WebSocket(STOMP)[11] 실시간 시세를 구현한다.
- AI 투자자 봇
  - 페르소나별 봇을 구현하고 스레드 풀로 운용하며, 주문 속도 제한을 둔다.
- LLM 연동
  - 프롬프트 구성, JSON 응답 검증, 24시간 호출·반영, 신규 반영 비율을 구현한다.
- 이벤트 엔진
  - 종속/비종속 사건, 연간 일정 생성, 사건 작용값 반영을 구현한다.
- 대회·랭킹
  - 기간 구간 관리, 일별 스냅샷, Redis[10] Sorted Set 순위 집계를 구현한다.
- React 프론트엔드
  - 시세 차트, 호가창, 주문, 랭킹 화면을 구현하고 WebSocket 구독으로 실시간 갱신한다[12].
- 시장 검증 및 부하 테스트
  - 변동성 군집과 두꺼운 꼬리 분포를 측정하고[6][7], 부하 시험으로 처리량과 파라미터를 조정한다.
- 위험 요소 및 대응
  - LLM 응답 지연·오류: 응답 검증, 재시도, 실패 시 이전 파라미터 유지, 호출 기록 보관
  - 가격 폭주·붕괴: 일일 가격제한폭(±30%)[2], 1회 파라미터 변경 폭 제한, 신규 반영 비율 조절
  - 엔진 장애: 주문·체결 로그 재생으로 호가창 복구
  - 봇 주문 폭증: 주문 속도 제한, 종목 단위 처리 분리, 부하 테스트
  - 유동성 부족: 마켓메이커형·노이즈 트레이더 봇으로 호가 유지
  - 비현실적인 가격 움직임: 통계적 특징 측정[6][7]으로 사건 확률과 파라미터 조정
  - 중도 참가자 형평성: 구간별 참가 시작일을 랭킹에 함께 표시

5. 관련기술

- 가격·시간 우선 호가 매칭

한국거래소 정규장은 연속매매 방식이며 가격 우선·시간 우선 원칙으로 체결한다[2]. 지정가 주문 장부의 구조와 통계적 특성은 학술적으로 정리되어 있으며[5], 본 프로젝트의 체결 엔진은 이를 참고하여 지정가·시장가 주문을 처리한다.

- 에이전트 기반 시장 시뮬레이션

여러 유형의 거래 에이전트가 각자의 규칙으로 주문을 내고, 그 상호작용으로 가격

이 형성되는 시뮬레이션 방식이다. ABIDES는 이산 사건 시뮬레이터 위에서 여러 거래 에이전트가 거래소 에이전트를 통해 거래하는 구조를 제시한다[4]. 본 프로젝트의 AI 투자자 봇은 이 접근을 따른다.

- JNI와 FFM API

JNI는 Java가 네이티브 코드와 상호작용하기 위한 표준 인터페이스이다[9]. JEP 454의 Foreign Function & Memory API는 JDK 22에서 정식 기능으로 제공되며[8], 네이티브 코드를 호출하는 또 하나의 방법이다. 체결 엔진 연동 시 두 방식을 비교한다.

- LLM 구조화 출력과 검증

LLM에게 정해진 형식(JSON)으로 파라미터를 응답하게 하고, 서버가 형식·범위·변동폭을 검증한 뒤 반영한다. LLM의 응답을 그대로 신뢰하지 않고 시장 안정성을 보호하는 구조이다.

- 시장 사실성 검증

실제 주가 수익률의 통계적 특징(stylized facts)은 Cont(2001)가 정리했다[6]. 후속 연구는 Cont의 11개 사실 중 수익률의 자기상관 부재, 두꺼운 꼬리, 변동성 군집, 절대 수익률 자기상관의 느린 감소 등을 최근 주식시장 데이터에서 다시 확인했다[7]. 가상 시장이 이러한 특징을 재현하는지 측정하고, 그 결과로 사건 확률과 파라미터를 조정한다.

- Redis Sorted Set

Redis의 Sorted Set은 점수 순 정렬을 유지하며, 순위 조회와 갱신이 O(log N)이고 동점은 멤버의 사전순으로 구분된다[10]. 본 프로젝트는 진행 중인 기간 대회의 수익률 순위를 이 구조로 집계한다.

- WebSocket과 STOMP

WebSocket은 하나의 TCP 연결에서 양방향 통신을 제공하며, 저지연·고빈도·대용량 메시지가 필요한 금융 서비스처럼 실시간성이 높은 환경에 적합하다. Spring은 WebSocket 위에서 STOMP 하위 프로토콜을 지원한다[11].

- React

React는 컴포넌트 단위로 사용자 인터페이스를 구성하는 라이브러리이다[12]. 본 프로젝트는 시세 차트, 호가창, 주문 입력, 랭킹 표를 컴포넌트로 구성하고, WebSocket 구독으로 받은 데이터를 화면에 반영한다.

6. 기대효과

- 가격 형성 과정의 체험

사용자의 주문이 가격에 영향을 주고, 같은 사건에도 투자자 유형별 반응이 달라지는 환경에서 수요·공급에 따른 가격 형성과 기대치 대비 결과를 고민하며 투자 판단을 연습할 수 있다.

- 상시 참여 가능한 기간별 대회

1개월·3개월·6개월·1년 단위의 논리적 대회에 자동 편입되므로, 원하는 때에 참여해 같은 기간의 다른 참가자와 수익률로 실력을 겨룰 수 있다.

- 재현 가능한 시장 환경

가상 기업과 가상 사건을 사용하므로 다양한 시장 상황을 의도적으로 연출할 수 있고, 모든 LLM 호출과 파라미터 변경 이력이 기록되어 시장 변화를 추적하고 재현할 수 있다.

- 확장성

시장 단위로 엔진과 봇을 분리하여 이후 여러 시장으로 확장할 수 있다.

7. 참고문헌

※ URL은 2026.10.12 기준으로 접속하여 확인한 자료이다.

[1] 한국투자증권, 모의투자안내 - 주식/선물옵션 모의투자 (증권사 모의투자 서비스 방식)
https://securities.koreainvestment.com/main/research/virtual/_static/TF07da010000.jsp

[2] 한국거래소(KRX), Guide to Trading in the Korean Stock Market (정규장 연속매매의 가격·시간 우선 원칙, 주문 유형, 일일 가격제한폭 기준가격 대비 ±30%, 매매 시간 09:00~15:30)
https://global.krx.co.kr/contents/GLB/01/0109/0109000000/guide_to_trading_in_the_korean_stock_market.pdf
KRX Global, Trading Hours
https://global.krx.co.kr/contents/GLB/06/0602/0602020204/GLB0602020204T1.jsp

[3] 한국거래소, KRX 정보데이터시스템(KRX Data Marketplace) 투자자별 거래실적 (개인·기관·외국인 투자자 유형별 거래 통계)
https://data.krx.co.kr/contents/MDC/MDI/outerLoader/index.cmd?screenId=MDCSTAT022&locale=ko_KR&kosdaqGlobalYn=1

[4] D. Byrd, M. Hybinette, T. H. Balch, "ABIDES: Towards High-Fidelity Multi-Agent Market Simulation", Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (SIGSIM-PADS '20), pp. 11-22, 2020. (arXiv:1904.12066) (이산 사건 기반 다중 에이전트 시장 시뮬레이션 구조)

https://arxiv.org/abs/1904.12066
[5] M. D. Gould, M. A. Porter, S. Williams, M. McDonald, D. J. Fenn, S. D. Howison, "Limit Order Books", Quantitative Finance, 2013. (arXiv:1012.0349) (지정가 주문 장부의 구조·통계적 특성·모형 조사)
https://arxiv.org/abs/1012.0349
[6] R. Cont, "Empirical properties of asset returns: stylized facts and statistical issues", Quantitative Finance, 1(2), 223-236, 2001. DOI: 10.1080/713665670 (실제 자산 수익률의 통계적 특징)
https://ideas.repec.org/a/taf/quantf/v1y2001i2p223-236.html
[7] E. Ratliff-Crain, C. M. Van Oort, J. Bagrow, M. T. K. Koehler, B. F. Tivnan, "Revisiting Cont's Stylized Facts for Modern Stock Markets", arXiv:2311.07738 (Cont의 11개 사실 중 8개가 최근 데이터에서 확인됨)
https://arxiv.org/abs/2311.07738
[8] OpenJDK, JEP 454: Foreign Function & Memory API (JDK 22 기능)
https://openjdk.org/jeps/454
OpenJDK, JDK 22 (2024.03.19 GA)
https://openjdk.org/projects/jdk/22/
[9] Oracle, Java Native Interface Specification (Java SE 21)
https://docs.oracle.com/en/java/javase/21/docs/specs/jni/design.html
[10] Redis, Redis sorted sets (랭킹 용도, ZADD/ZRANK/ZREVRANK의 O(log N) 복잡도, 동점 시 사전순)
https://redis.io/docs/latest/develop/data-types/sorted-sets/
[11] Spring Framework Reference, WebSockets (WebSocket과 STOMP 지원, 저지연·고빈도·대용량 환경에 적합)
https://docs.spring.io/spring-framework/reference/web/websocket.html
[12] React, The library for web and native user interfaces
https://react.dev/